Technical report — Abstimmungsbüchlein Check: NLI over Swiss federal voting booklets with Apertus

What we built, how it works, and what the numbers say. Every number comes from a recorded run in `docs/runs/` (short names in brackets, listed under References) on the dev split, the validation set (val), the probe set or all three; **the sealed test dates were run once, after the freeze (section 1)**.

- **Track:** Track 2A — OST: Multilingual Natural Language Inference over Swiss Official Voting Booklets
- **Event:** Online
- **Project:** Abstimmungsbüchlein Check — multilingual claim verification against Swiss voting booklets
- **Team:** Bissegger & Bayraktar — Rashad Bissegger, Kaan Bayraktar
- **Repository:** https://github.com/Rashad18236/hackapertus-voting-nli
- **Image:** ghcr.io/rashad18236/hackapertus-voting-nli:latest (digest `sha256:0c1a10cd92261cdfede7eb8c191a16eab3f94770f0a7386b45a06ebc06f088be`, commit `71a6d6a`)
- **Demo:** none

1. Summary

Given a claim in German, French or Italian and a source from an official Swiss federal voting booklet, the system decides whether the source supports (entailment, 0), does not settle (neutral, 1) or refutes (contradiction, 2) the claim. The source is a passage (task B) or a whole booklet PDF (task A). **Apertus v1.5 makes every decision.** Everything else is local and runs on CPU in the Docker image: reading the PDF, choosing the part of the booklet Apertus reads, and turning the paragraphs Apertus cites into evidence quoted verbatim with their 1-based PDF page. Apertus also names a claim's part when the rules cannot, reads a booklet's contents when our parser cannot, translates a claim for the fallback's keyword search, and takes a second look at cited paragraphs after a neutral answer (section 4).

The central experiment, full booklet against selected context, decided the design: on 100 dev cases [E1-curve] the whole booklet scored 0.858 with 37,586 input tokens per case, the part the claim names (`section-route`) 0.989 with 1,248; without booklet text Apertus scores 0.435 [E2-cb].

Final defaults through the published image [FIN]	dev	val	probe
Task A Macro-F1 (weight 0.30 of the score)	**0.977**	**0.981**	**0.990**
Task A evidence (0.12)	**0.990**	**0.988**	**1.000**
Task A tokens per case, input + output (K, 0.09)	1,259 + 14.9	1,233 + 14.9	1,177 + 16.6
Task A local time per case, model at 1.0 / 0.3 s per request (T, 0.09)	0.020 / 0.027 s	0.010 / 0.010 s	0.039 / 0.035 s
Task B Macro-F1 (0.28)	**0.987**	**0.968**	**0.977**
Task B tokens per case, input + output (K, 0.06)	815 + 9.0	820 + 9.3	789 + 8.3
Task B local time per case, 1.0 / 0.3 s per request (T, 0.06)	0.012 / 0.021 s	0.009 / 0.010 s	0.020 / 0.019 s
Labels written by Python (neutral fallback)	0 of 600	1 of 1,160	0 of 188
REF SCORE at 1.0 / 0.3 s per request (section 2)	**0.904 / 0.893**	**0.908 / 0.907**	**0.901 / 0.902**

All from the session 15 image `sha-7ae78ad87cfd` (A6 on); dev local time: mean of two. Three more dev runs gave the same 600 labels (§ 6).

Sealed test dates (2020-02-09, 2022-02-13, 2022-05-15): TODO, results after the organisers' evaluation.

2. What is scored, and what we optimise

The organisers' score (Q&A of 8 October; `docs/official_scoring.md`): **Score = 0.4 S_B + 0.6 S_A**, with S_A = 0.50 Macro-F1 + 0.20 evidence + 0.15 T + 0.15 K and S_B = 0.70 Macro-F1 + 0.15 T + 0.15 K. K = the best team's tokens per case ÷ ours; T = the best local time per case ÷ ours, where **local time is the container's wall-clock time minus the time during which a model request is in flight**: start-up, PDF reading, embedding and every gap between two requests count. Task A and task B run in separate containers. To rank our own versions we use **REF SCORE** (`scripts/official_score.py`): the same formula with "best" values fixed by us before any measurement (600 / 800 tokens, 0.05 / 0.005 s local time per case for task A / B). Local time is measured the organisers' way (`scripts/local_time.py`): the image with 2 CPUs and 4 GB against a fake model that replays saved answers after 1.0 s (about Public AI's speed), 0.3 s or 0.1 s per request.

3. Architecture

```
cases.jsonl ─► src/cli.py ─► src/pipeline.py: worker processes ("lanes") prepare cases
──► ready queue
  task B: the reference (long: first line + 4 paragraphs) ─► prompt
──────────────────────┐
  task A: booklet PDF ─► src/parse.py (pypdf, pages read when first needed, 1-based)
│
          ├─ section-route: src/claim_router.py (part named by the opening)
│
          │  + src/booklet.py (votes, parts, checked headings) ─► numbered paragraphs
──┤
          ├─ opening unknown to the rules: Apertus names the part (one call) ─► same
──┤
          ├─ parser finds no valid part: Apertus reads the contents (one call per
booklet) ┤
          └─ fallback: 8 chunks by BM25 (Apertus translates the claim across languages)
─┤

▼
       main process: src/llm.py ─► BASE_URL (Apertus v1.5), one request at a time, one
kept connection
  predictions.jsonl ◄─ evidence: cited paragraphs, verbatim, with page
◄─────────────────────┘
```

- `src/cli.py`: the entrypoint; each case is `prepare()` (local work) then `complete()` (model calls); every id is answered (neutral if all else fails), each answer written at once, exit code 0.
- `src/pipeline.py`: two forked lanes, each owning whole booklets, prepare cases while the main process sends them; one lane under 2.5 GiB when a case can load e5 (task B), under 512 MiB for task A with model S. A **memory guard** (`src/rankers.py`) does without an embedding model whose need (e5: 1.5 GiB) exceeds the container's limit or that fails to load: long parts send their first 8 paragraphs, long task B references go whole. Caches are keyed by content; replaying the saved answers of all 1,180 dev and val requests reproduces every request, label and evidence (G1, G2).
- `src/llm.py`: the only module that calls a model (`BASE_URL`, temperature 0, one kept connection, retries).

3.1 How document context is prepared and supplied to the model

Parsing. pypdf reads a page's text when first needed, with its 1-based PDF page number; an unreadable page is repaired or left empty, never failing the booklet.

Selection (`section-route`, task A). (1) *Router:* the claims open with a formula naming their source ("Laut der Zusammenfassung …", "Le Conseil fédéral estime …"); regular expressions map it to one of five parts: summary, Federal Council, committee, text put to the vote, detailed explanation. If they find none, Apertus names the part (one call). (2) *Parser:* the table of contents gives each vote's sections (two layouts: before and after mid-2018); every start page is checked against its printed heading; the arguments are split by voice. If no part of the vote passes, Apertus reads the contents pages (one call per booklet) and its start pages go through the same checks. (3) *Vote match:* the case's `vote` against the contents titles (fuzzy); if that fails, by meaning (model

S). (4) *Paragraphs:* the part's lines grouped into paragraphs, each with its page; a part over 8,000 characters keeps its 8 paragraphs most similar to the claim (model S, a static multilingual embedding; e5 until session 13). (5) *Fallback:* no part, no vote, or any error → the booklet's 8 chunks of up to 1,000 characters with the best BM25 score for the claim's words; a claim in another language than the booklet is first translated into the booklet's language by Apertus (one call), and the reading call gets the claim as it is.

What the prompt contains. System: the instruction and the decision rule (`A-v4-section-route-L1`):

> *Decide in this order: 1. Does the reference text deal with the subject of the claim at all? If not, the label is 1 (neutral). 2. If it does: 0 if it supports the claim; 2 only if it states something that cannot be true together with the claim; otherwise 1. Missing information is never a contradiction. […]*

User: `PART:` (which part, whose voice), `REFERENCE TEXT:` with paragraphs `[1] … [n]`, `VOTE:`, `CLAIM:` as given. `response_format: json_schema` forces `{"paragraphs": [≤ 3 numbers], "label": 0|1|2}`. Task B sends the same rule (`v3-topic-first`) for `{"label": n}`; a reference over 3,000 characters is sent as its first line and the 4 paragraphs whose first 300 characters are most similar to the claim (e5). **A6** (task A, since session 15): when the answer is neutral but cites paragraphs, one more request sends only the cited paragraphs with their neighbours and the claim (`A-v6-second-look`) and asks for `{"finding": "same" | "different" | "nothing"}` (→ 0 / 2 / 1).

Evidence. The paragraphs Apertus cited, verbatim with their page, then their halves, then (H, session 13) the paragraph before the first cited one when that one opens its page, and the cited page's whole text; at most five items, every one on its real page; none for neutral answers and task B.

4. How Apertus is used

The organisers (`docs/official_contract.md`): "Use only **Apertus v1.5** models (`swiss-ai/Apertus-v1.5-...`)" and "Every remote model call must use `BASE_URL`; local parsing, OCR, or embeddings may run locally." **Apertus makes every judgement; Python only decides what Apertus reads.** Python and the local embedding model read the PDF, pick the part, and copy what Apertus cites; they never set or change a label (the only Python label is the neutral fallback for an unreadable answer: 1 of 1,948 final labels). Model `swiss-ai/apertus-v1.5-8b` on Public AI, zero-shot, temperature 0, answers constrained by `json_schema`, only through `BASE_URL`; no fine-tuning.

Role of Apertus (final dev run [FIN])	Called when	Calls per 100 cases (A / B)	Tokens per call, in + out
Label and the paragraphs it rests on	every case	100 / 100	1,251 + 14.6 (A); 815 + 9.0 (B)
AP-part: the part a claim's opening names	the rules know no part for the opening	0 / –	≈ 236 + 49
AP-structure: read the booklet's contents	the parser finds no valid part	0 / –	≈ 1,825 + 192 per booklet
AP-translate: the claim into the booklet's language	fallback across languages	0.33 / –	141 + 55
A6: a second look at the cited paragraphs and their neighbours (finding same / different / nothing → 0 / 2 / 1)	the label is neutral but cites paragraphs	1.7 / –	454 + 8

Ablation, task A dev (minimum Macro-F1 0.60; task B 0.75)	Macro-F1	Evidence	Input tokens per case
nothing: claim and vote only [E2-cb]	0.435	—	≈ 200
8 chunks found by embedding, the early baseline [E5]	0.834	0.662	1,868
the whole booklet, 100 cases [E1-curve]	0.858	—	37,586
Apertus does all parsing too (part and contents, every case) [AP-all]	0.960	0.891	1,655 (+34 %; local time 8×)
the part the claim names (routed), with A6 [FIN]	**0.977**	**0.990**	1,259

A larger Apertus could not be measured as a reference: Public AI lists `swiss-ai/apertus-v1.5-70b`, but on 15 October it refused all 9 calls we sent it over 41 minutes (21 HTTP requests, every one HTTP 429) [S15b].

Errors (session 12's val answers, `scripts/error_types.py`): of 23 task A errors, **10 are routing errors** (gold passage not read; 8 in long parts cut to 8 paragraphs) and **13 reading errors** (direction or negation 6, scope 5, number or date 2, as Apertus names them offline); all 18 task B errors are reading errors (another ballot or topic 6, number or date 6, scope 4, other 2).

Tried, not adopted (each by a rule fixed before its interleaved run; all in the code, off)	Result against today (Macro-F1, dev unless marked)	Why not
short prompts P-short (A), BP-short (B); three examples in the prompt (B)	0.670; 0.667; 0.933 vs 0.947	lower
K4: the part cut to its 4 most similar paragraphs	dev 0.990 (−35 % tokens); val 0.944 vs 0.959	fails on val
claim decomposition / tournament across languages / two-call negation	A 0.963 / 0.980 / 0.987 vs 0.983; B 0.937 / 0.970 / 0.960 vs 0.967	lower, 1.7–3 calls per case
L2 (second look with the 3 most similar paragraphs)	dev +0.010, val +0.003	below its +0.015 bar
task B as numbered paragraphs (B-para); B-cut4	0.963 vs 0.967; −55 % tokens but slower	fail their rules
task B selectors B-R / H / B-BM25	0.973 / 0.993 (+13 % tokens) / 0.967 vs 0.987	fail their rules
Python rule "neutral with a citation → contradiction"	val +0.014 offline; on-topic neutral 3 of 3 wrong	Python may not set a label
session 13 prompts A1–A5 against contradictions answered neutral	dev 0.902–0.983 vs 0.983; on-topic neutral 13–28 of 46 vs 32	fail on dev
B1 (ballots first; session 15: shorter fields) / B3 (T with the title) / B4 (claim first)	B1 void, retry 0.892 (19 unreadable); 0.980 / 0.980 vs 0.987	lower or unreadable
A6b: A6 told that "related but not settled" is "nothing"	same finding in all 19 second looks	on-topic flips 3 vs 3
B6: after an answer 2, "is the claim about the ballot in the title?"	0.970 vs 0.987; 1 fix, 6 breaks; +7 % tokens	misstated ballots called "not about it"

Who did what. Rashad Bissegger: routing (claim router, booklet parser, `section-route`), evidence (cited pieces, halves, hedge H), timing and memory (lanes, memory guard), model S, A6, and the session method (a rule fixed before every interleaved run) [FIN, COV, GRID, GUARD, STAB]. Kaan Bayraktar: the BM25 and translated-claim retrievers that became the fallback [K, S12], the finding that a claim's and its negation's answers disagree on almost every error (all 10 task B and 4 of 5 task A dev errors, in about 7 % of cases), and the tournament and decomposition experiments [K].

5. Data

- **Dataset:** `OSTswiss/MNLIoverSwissVotingBooklets` v1.1 (commit `9ff08597`, MIT): 1,488 rows, 335 exact duplicates; each row gives one task A and one task B case.
- **Splits** (by voting date, duplicates removed): test 5 dates (267 rows); dev 300 rows; val 580 rows of the dev dates (never used to change a rule); **probe:** the 94 test rows of 2021-03-07 and 2025-09-28; **sealed:** the 173 test rows of the three other dates, opened once after the freeze (section 1).
- **Booklets:** the Federal Chancellery's PDFs, downloaded by script and not committed. For parser coverage, every federal booklet from 2016 to 2026 in de, fr and it (93 PDFs; the sealed dates never downloaded). Swiss copyright law exempts official reports of authorities (Art. 5 URG). No personal data.

6. Evaluation

Method. The starter's `evaluate.py`, unchanged. The endpoint drifts, so **only arms of one interleaved run compare** (every case answered by all versions back to back; all session 11 and 12 answers came from one backend). Rules were fixed before the runs; a version that fails stays in the code, switched off.

Endpoint drift and stability [STAB]. Public AI's 8B changed its answers to the same requests twice on 8–9 October, served two backends, and answers a repeated request from a cache for about ten minutes. To measure the noise of the final defaults, the final image answered all 600 dev cases three times, each run 30 minutes after the previous one (past the cache): **0 of 600 labels changed** (task A 0.977, task B 0.987 each time; one backend; 10 task A answers cited other paragraphs, 20 task B answers carried trailing text). Session 9's three runs over 16.5 hours

changed 2 of 600. Run-to-run noise is far below the sampling error of 300 cases: 95 % bootstrap intervals (2,000 resamples) task A [0.958, 0.991], task B [0.973, 0.997]; differences of that size between dev, val and the test set are sampling noise.

Parser coverage [COV] (93 booklets 2016–2026; the three languages of a date must agree): booklets whose votes all pass 64 → **88**, votes passing every check 178 → **244 of 249**, probe cases routed 83 → **90 of 94** (session 11).

Fallbacks, forced [FB-dev, FB-val, S12-dev, S12-val]: every case put into a fallback situation by a test-only switch (“no part”: the router returns nothing, AP-part off; “no vote”: the vote match fails; “no parser”: none of the vote’s parts passes its checks). Macro-F1 / evidence, dev; val (balanced 300); each line against the arm of its own interleaved run:

Situation, version	dev	val	Tokens (dev)	Adopted?
routed (not forced, session 12)	0.977 / 0.980	—	1,238 + 14.5	
no part: BM25 chunks, Apertus translates across languages	**0.956 / 0.736**	**0.957 / 0.755**	2,591 + 44.3	**yes**
no parser: Apertus reads the contents	**0.946 / 0.925**	**0.939 / 0.917**	1,564 + 42.8	**yes**

Both replaced e5’s 8 chunks of the booklet (no part 0.900 / 0.672, no parser 0.896 / 0.672 on dev). Not adopted: vote-wide (0.927 / 0.873) and booklet-wide paragraphs (0.857 / 0.856); model S’s chunks (0.913, dev).

Task B, fewer tokens [TB-dev, TB-val]: selector T (section 3.1) scores dev 0.987, val 0.968 with 815 input tokens per case, against 0.967, 0.954 with 1,994 for the whole reference.

Local time (the organisers’ measure; image, 2 CPUs, fake model replaying the final answers) [FIN]. What is local in task A is PDF reading (about 1.3 s per booklet, pure-Python pypdf) and the ranking of long parts. With e5 (until session 12) the ranking took about 3 s per long part and stopped hiding behind fast requests; model S (session 13) ranks in milliseconds:

Local time per case, dev	1.0 s per request	0.3 s	0.1 s
Task A, model S, 4 GB (final image, session 15; mean of 2)	0.020 s	0.027 s	0.062 s
Task A, model S, 1 GB: two lanes (session 14) / one lane (session 13’s code)	0.019 s / 0.020 s	0.024 s / 0.019 s	**0.063 s / 0.211 s**
Task A, e5 (session 12’s final image)	0.017 s	0.160 s (repeat 0.113 s)	0.364 s
Task B (e5 for selector T; session 15, mean of 2 / session 13)	0.012 / 0.008 s	0.021 / 0.009 s	0.172 / 0.132 s

Single measurements vary by up to about ±0.03 s at 0.3 s and with the machine’s state: task B’s code is unchanged since session 11, yet its two sessions’ rows differ. The 1 GB row compares both lane versions in the same rounds (`lanes-S_devA300`).

Memory [GRID, GUARD]: task A peaks at about 0.2–0.4 GB (with e5 in session 12: 2,287 MiB) and keeps two lanes down to 512 MiB; task B peaks at 1.2–1.7 GB (e5), one lane under 2.5 GiB, no e5 below 1.5 GiB (long references go whole). The final image writes every response of both tasks at 4g, 2g, 1.5g, 1g, 768m and 512m, with 2 CPUs and with 1.

Probe: two unseen voting dates (test rows of 2021-03-07 and 2025-09-28, opened in session 10). Through the final image [FIN]: task A **0.990**, evidence **1.000** (65 of 65), task B **0.977**; REF SCORE **0.901 / 0.902**. Session 13’s image (A6 off): 0.981, 0.985, 0.977. 90 of 94 task A cases are routed; the 4 others are claims about a committee in votes that have none, answered by the BM25 fallback.

7. Limitations

- **Tuned on dev.** Prompts, router patterns and parser checks were written with dev in view; val confirms; the probe (two unseen dates) lands at val’s level.
- **`section-route` depends on the booklet layout** (2016–2026 checked: 244 of 249 votes pass). A claim whose opening neither the rules nor Apertus place, or a vote the parser cannot read,

falls back (answered, less accurate).

- **The 8B.** The submission calls Apertus v1.5 8B: the organisers' endpoint and model are unknown to us, every rule, prompt and threshold was built and measured on the 8B, and the one larger v1.5 model listed (70B) did not answer.
- **Endpoint and speed.** Our scores hold for the backend that answered; with a fast model, PDF reading no longer hides behind the requests (model S removed e5's share in session 13).
- **A6's known weakness.** "A6 failed its on-topic condition twice by one case (2 of 46 in session 13, 3 of 46 in session 14) and passed every other condition in both runs (val +0.014 and +0.012, probe +0.010, 19 fixes against 2 breaks on 1,254 real cases). The team adopts it on the strength of val and probe and records the on-topic result as its known weakness: a passage on the claim's subject that does not settle the claim can be answered 'different'." (From the run files: 19 against 2 includes session 13's dev run; on the 1,254 val and probe cases, 18 against 2.)
- **Task B: claims about another ballot.** 8 of the final val run's 19 task B errors are unrelated passages called contradictions; three attempts to catch them (B1, its retry, B6) cost more than they fixed (section 4).
- **Evidence pages.** If the official run checks the gold passage's *first* page, about 40 % of our hits would not count (the passage often starts a page earlier); H recovers part of it (`docs/organiser_questions.md`).
- **Not political advice.** The system checks a claim against the booklet's text only. Its answers are not voting recommendations. Every task A label 0 or 2 is traceable to the booklet: it comes with the page and the verbatim text it rests on.

8. Reproducibility

- **Published image:** `ghcr.io/rashad18236/hackapertus-voting-nli:latest` (built from `main` by the publish workflow, tested on a fresh runner), run with the organisers' command (README). The sealed run used digest `sha256:0c1a10cd92261cdfede7eb8c191a16eab3f94770f0a7386b45a06ebc06f088be` (commit `71a6d6a`); the final dev, val and probe numbers come from `sha-7ae78ad87cfd` (the same `src/`).
- **Submission tag:** `submission` (set on main after this PR).
- **From source:** `make run` builds the image and answers `examples/cases.jsonl`.
- **Hardware:** CPU only; measured with 2 CPUs and 4 GB; every case answered down to 512 MiB, 2 CPUs or 1 [GRID].
- **Determinism:** temperature 0; a fake model replays saved answers by request hash (G1, G2); 358 tests.
- **License:** code Apache-2.0 (the repository's `LICENSE`); report text CC-BY-4.0.

References

- Dataset: OSTswiss/MNLIoverSwissVotingBooklets v1.1, Hugging Face (MIT). Starter: `gitlab.com/ifsoftware/hackapertus-starter`. Apertus v1.5, Swiss AI Initiative.
- Wang, L. et al. (2024). *Multilingual E5 Text Embeddings.* arXiv:2402.05672 (MIT). Model S: `static-similarity-mrl-multilingual-v1` (Apache-2.0). Robertson, S., Zaragoza, H. (2009). *BM25 and Beyond.*
- Runs (`docs/runs/`, dated 2026-10-08 to 15): FIN `final-s15_*`; STAB `stability-s15-run{1,2,3}_dev600`; GRID `memory-grid-final-s15_grid24`; GUARD `memory-guard_grid48`; TB `taskb-selectors_*`; COV `parser-coverage_booklets93`; FB `fallbacks_*`; S12 `S-and-fallbacks_devA300`, `bm25ap-fallback_valA300`, `ap-structure-ask_*`; E1-curve `context-curve_devA100`; E2-cb `closed-book_devA300`; E5 `section-route-vs-embed_devA300`; AP-all `ap-all_devA300`; not adopted (section 4): `prompts-A_*`, `prompts-B_*`, `A6b_*`, `B1f_devB300`, `B6_devB300`; S15b `70B-smoke_devA3-devB3`; K `kaan_strategies_dev600`, `kaan_retrievers_*`; SEALED `sealed-once`.